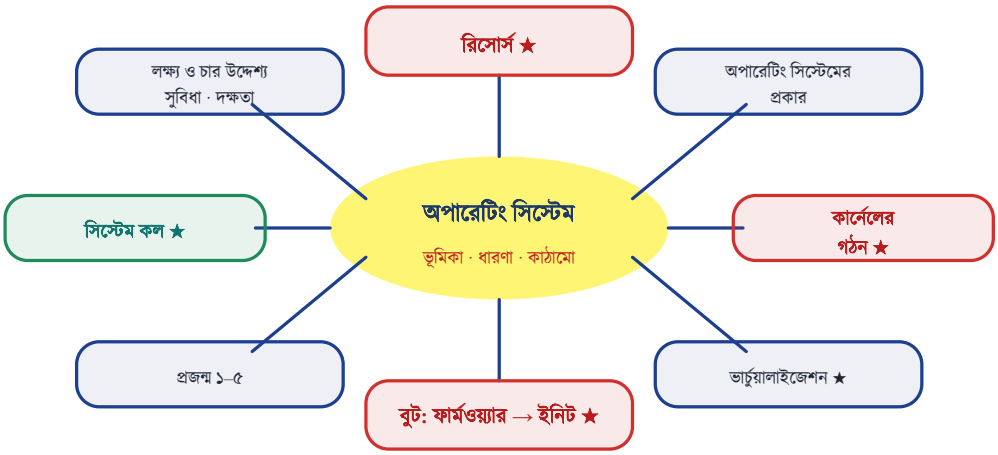
অপারেটিং সিস্টেম

লক্ষ্য · সিস্টেম কল · কার্নেলের গঠন · বুট · ভার্চুয়ালাইজেশন: ১২ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = OS-এর মাঝের সেই একটি দরজা

সিলেবাসের যে অংশ

অপারেটিং সিস্টেমের সিলেবাসের প্রথম দুই লাইন "**কম্পিউটার সিস্টেমে এর ভূমিকা**" এবং "**OS-এর ধারণা; OS-এর কাঠামো**"। এই নোটবুক ঠিক সেই দুটিই আড়াল করে; প্রসেস, শিডিউলিং, IPC, মেমোরি, I/O, ডেডলক ও ফাইলসের নিজস্ব লেসন আছে।

হাই-ইল্ড তালিকা

① OS = ইন্টারফেস + ম্যানেজার + কন্ট্রোল
② লক্ষ্য: সুবিধা, দক্ষতা, বিবর্তন, পোর্টেবিলিটি
③ কার্নেল প্রিভিলেজড; শেল কেবল একটি UI
④ কল ⇒ ইউজার → কার্নেল, ট্র্যাপ-এর মাধ্যমে
⑤ মনোলিথিক: সব কিছু কার্নেল স্পেসে
⑥ মাইক্রোকার্নেল: শুধু IPC ও শিডিউলিং
⑦ হাইব্রিড: Windows NT, macOS XNU
⑧ টাইপ ১ = বেয়ার মেটাল, টাইপ ২ = হোস্টের ওপরে
⑨ বুট: ফার্মওয়্যার → লোডার → কার্নেল → ইনিট
⑩ কনটেক্সট সুইচ ≈ ১–১০ µs

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

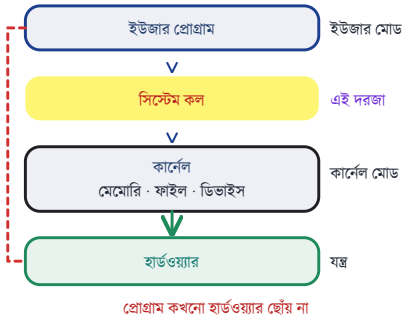
সূচিপত্র

✓ অপারেটিং সিস্টেম কী: লক্ষ্য ও ভূমিকা … পৃ. ২
✓ সিস্টেম কল ও মোড সুইচ … পৃ. ৩
✓ কার্নেলের গঠন: মনোলিথিক → মাইক্রো … পৃ. ৪
✓ কার্নেল, শেল ও ইউটিলিটি … পৃ. ৫
✓ অপারেটিং সিস্টেমের প্রকার … পৃ. ৬
✓ অপারেটিং সিস্টেমের প্রজন্ম … পৃ. ৭
✓ VM গঠন ও হাইপারভাইজর … পৃ. ৮
✓ বুট: ফার্মওয়্যার → ইনিট … পৃ. ৯
✓ ট্র্যাপ, ইন্টারাপ্ট ও সুইচের খরচ … পৃ. ১০
✓ রাতের পরীক্ষার পাতা … পৃ. ১১
✓ অনুশীলনী MCQ + লিখিত প্রশ্ন … পৃ. ১২

১ অপারেটিং সিস্টেম কী?

সংজ্ঞা: **অপারেটিং সিস্টেম** হলো ব্যবহারকারী ও হার্ডওয়্যারের মাঝের সিস্টেম সফটওয়্যার, যা একসঙ্গে তিনটি কাজ করে: একটি **ইন্টারফেস** (প্রোগ্রাম সরাসরি ডিভাইস চালায় না, সেবা চায়), একটি **রিসোর্স ম্যানেজার** (CPU, মেমোরি, ফাইল ও I/O ন্যায্যভাবে ভাগ হয়) এবং একটি **কন্ট্রোল প্রোগ্রাম** (হার্ডওয়্যারে প্রবেশ কেবল প্রিভিলেজড, তাই কেবল OS-ই পারে)। একই বস্তুর দুটি বইয়ার নাম: **extended machine** এবং **resource-manager system**।

OS ছাড়া = শুধু একটি সার্কিট



একটি কল = একটি সীমানা পার হওয়া

একই প্রোগ্রামের দুটি ধরনা: **রিসোর্স ম্যানেজার / কন্ট্রোল প্রোগ্রাম** — কৌকুটির উত্তর: *কে কখন কী পাবে*: CPU সময় শিডিউলিংয়ে, মেমোরি অ্যালোকেশনে, ফাইল ফাইল সিস্টেমে, ডিভাইস কন্ট্রোলার ও ড্রাইভারে। এটি **দক্ষতা** বাড়ায়।

Extended machine — প্রোগ্রাম কী দেখে তার উত্তর: ফাইল, প্রসেস ও ডিভাইসের একটি বিমূর্ত যন্ত্র; নিচে ইন্টারাপ্ট, DMA ও ক্যাশের নিয়ম লুকানো। এটি **সুবিধা** ও **পোর্টেবিলিটি** বাড়ায়।

লক্ষ্য	এটি কী চায়	কোথায় চেনা যায়
সুবিধা	ব্যবহারযোগ্য ইন্টারফেস, মেশিন কোড নয়	সিস্টেম কল, CLI শেল, GUI, ড্রাইভার
দক্ষতা	রিসোর্স পূর্ণ ক্ষমতায় ব্যবহার	শিডিউলিং, পেজিং, বাফারিং, ক্যাশিং
বিবর্তনের সামর্থ্য	নতুন হার্ডওয়্যার নতুন লেখা ছাড়াই গ্রহণ	মডুলারিটি, লেয়ার, লোডেবল মডিউল
পোর্টেবিলিটি	একই OS ভিন্ন হার্ডওয়্যারে	স্ট্যান্ডার্ড ভাষা (C), হার্ডওয়্যার অ্যাবস্ট্রাকশন লেয়ার

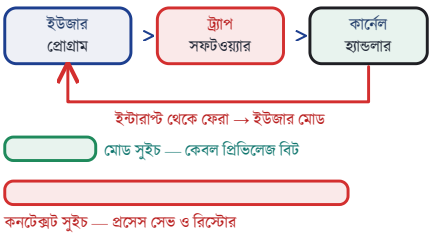
মনে রাখার কৌশল: চারটি G: **C**onvenience = *সহজ*, **E**fficiency = *পূর্ণ*, **E**volution = *বদলানোযোগ্য*, **P**ortability = *বাড়ি বদলায়*। "পেজিং কোন লক্ষ্য মেটায়?" → **দক্ষতা**: এটি ব্যবহার বাড়ায়, আরাম নয়। "হার্ডওয়্যার অ্যাবস্ট্রাকশন লেয়ার কোনটি?" → **পোর্টেবিলিটি**।

পরীক্ষার ফাঁদ: একটি **ড্রাইভার একটি ডিভাইস সামলায়; OS সব ডিভাইস সামলায় ও কাকে কী দেবে ঠিক করে**। আর "ইন্টারফেস" অস্পষ্ট: **শেল** ব্যবহারকারীর ইন্টারফেস, **কার্নেল** আসল OS। "কোন অংশটি প্রিভিলেজড চলে?" → **কার্নেল**, কখনো শেল নয়।

সমাধানকৃত উদাহরণ ✍️ একটি মেশিনে লক্ষ্যের প্রয়োগ: একটি CPU, ৮ GB RAM, একটি ডিস্ক, চারজন ব্যবহারকারী। **সুবিধা** — প্রত্যেকে একটি শেল ও নাম দেওয়া ফাইল পায়, ডিস্ক কন্ট্রোলার প্রোগ্রাম করতে হয় না। **দক্ষতা** — শিডিউলার টাইম-স্লাইস দিয়ে CPU কখনো খালি থাকে না; ফাইল সিস্টেম গরম ব্লক ক্যাশ করে। **বিবর্তন** — নতুন প্রিন্টারে একটি ড্রাইভার লাগে, নতুন OS নয়। "*ভার্চুয়াল মেমোরি* কোন লক্ষ্য মেটায়?" → **দক্ষতা**।

২ সিস্টেম কল, ট্র্যাপ ও মোড সুইচ

সংজ্ঞা: **সিস্টেম কল** হলো সেই ইন্টারফেস যার মাধ্যমে ইউজার প্রোগ্রাম কার্নেলের কাছে সেবা চায় — ফাইল খোলা, মেমোরি ম্যাপ করা, প্রসেস তৈরি, ডিভাইস পড়া। কার্নেলই কেবল **প্রিভিলেজড** কোড চালায়, তাই অনুরোধে CPU-র প্রিভিলেজ বদলাতেই হয়। কৌশলটি **ট্র্যাপ** (সফটওয়্যার ইন্টারাপ্ট): CPU কনটেক্সট সেভ করে **কার্নেল মোডে** যায়, হ্যান্ডলার চালায়, ফিরে আসে। শুধু প্রিভিলেজ বিট বদলালে তা **মোড সুইচ** — সস্তা; অন্য প্রসেসের অবস্থাও লোড করলে তা **কনটেক্সট সুইচ** — সস্তা নয়।



"সুইচ" লেখার ভিতরে দুটি ভিন্ন খরচ

সিস্টেম কলের গোষ্ঠী	উদাহরণ
প্রসেস নিয়ন্ত্রণ	fork, execve, wait, exit, kill
ফাইল কাজ	open, close, read, write, lseek
ডিভাইস ব্যবস্থাপনা	ioctl, ডিভাইসে read ও write
তথ্য	getpid, uname, sysconf, time
মেমোরি ব্যবস্থাপনা	brk, mmap, munmap

বিষয়	ইউজার মোড	কার্নেল মোড
কে চালায়	সব সাধারণ প্রসেস: শেল, এডিটর, কম্পাইলার	শিডিউলার, মেমোরি ম্যানেজার, ড্রাইভার, ফাইল সিস্টেম
মেমোরি প্রবেশ	শুধু নিজের অ্যাড্রেস স্পেস	**যেকোনো** অ্যাড্রেস স্পেস, MMIO রেজিস্টার, পেজ টেবিল
নির্দেশ	সীমিত সাবসেট; প্রিভিলেজড হলে ট্র্যাপ হয়	সম্পূর্ণ সেট (x86 ring 3 → 0)
কীভাবে ঢোকে	কল, ফল্ট বা ইন্টারাপ্ট থেকে ট্র্যাপ	— (return-from-interrupt দিয়ে বের হয়)

মনে রাখার কৌশল: কার্নেলে ঢোকার তিনটি দরজা: **ট্র্যাপ** (প্রোগ্রাম চাইছে), **ফল্ট** (CPU অভিযোগ করছে — পেজ ফল্ট), **ইন্টারাপ্ট** (পৃথিবী চাইছে — টাইমার, কীবোর্ড, ডিস্ক)। তিনটিই কার্নেল মোডে যায়, হ্যান্ডলার বেছে নেয় ভেক্টর নম্বর দিয়ে।

সমাধানকৃত উদাহরণ ✍️ ওভারহেড কত?:
একটি ফাংশন কল ≈ **২০ ns**; ট্র্যাপসহ সিস্টেম কল ≈ **১ µs**। নিজের কাজের প্রতি সেকেন্ডে $১০^৬$ সিস্টেম কল করা একটি প্রোগ্রাম তাই **একটি কোরের ১০০ %** কার্নেলের ভেতরে কাটায়।

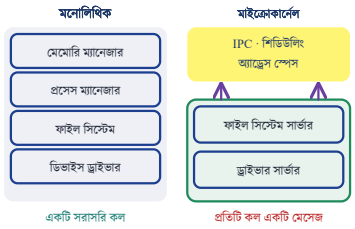
IT সংযোগ: `man 2 read` = *read* সিস্টেম কলের ম্যানুয়াল; `strace -c ls` ট্র্যাপ গোনে; `time ls` ইউজার ও সিস্টেম সময় আলাদা করে দেখায়।

পরীক্ষার ফাঁদ: সিস্টেম কল **আপনার প্রোগ্রামের দুটি ফাংশনের মধ্যে কল নয়**: লাইব্রেরি র‍্যাপার (glibc) **ইউজার মোডে** চলে, কেবল ট্র্যাপই কার্নেলে ঢোকে।

৩ কার্নেলের গঠন: মনোলিথিক, লেয়ারড, মাইক্রো, মডুলার, হাইব্রিড

সংজ্ঞা: **কার্নেল** হলো প্রিভিলেজড কোর: এটি কার্নেল মোডে চলে, মেমোরিতে স্থায়ী থাকে, হার্ডওয়্যারের মালিক এবং সিস্টেম কল বাস্তবায়ন করে। এর ভেতরের গঠন একটি নকশার সিদ্ধান্ত — **মনোলিথিক**, **লেয়ারড**, **মাইক্রোকার্নেল**, **মডুলার** — এবং তার মাঝামাঝি **হাইব্রিড**। সবই একটি প্রশ্নের উত্তর দেয়: **কতটা কোড প্রিভিলেজড হিসেবে বিশ্বাসযোগ্য?**

গঠন	কোথায় কী থাকে	শক্তি	দুর্বলতা	উদাহরণ
মনোলিথিক	সব কিছু কার্নেলের ভেতরে	গতি, সহজ কল-পথ	একটি বাগ সব থামায়	UNIX, Linux, MS-DOS
লেয়ারড	সব কিছু কার্নেলে, স্তরে স্তরে	যাচাই করা সহজ	স্তর পেরোনোর খরচ	THE, MULTICS, OS/2
মডুলার	কোর + লোডেবল মডিউল	নমনীয়, ছোট ইমেজ	মডিউলের বাগ আলাদা হয় না	Linux, Solaris, VxWorks
মাইক্রোকার্নেল	শুধু IPC, শিডিউলিং, অ্যাড্রেস স্পেস	নির্ভরযোগ্য, প্রমাণযোগ্য	মেসেজ ওভারহেড	Mach, MINIX 3, QNX, seL4
হাইব্রিড	বেশিরভাগ, মাইক্রোকার্নেলের ধরনে	গতি ও মডুলারিটি দুটোই	না সবচেয়ে সহজ, না সবচেয়ে নিরাপদ	Windows NT, macOS XNU



একই সেবা — শুধু দেয়াল সরে গেছে

পরীক্ষার ফাঁদ: **হাইব্রিড মানে "অর্ধেক-অর্ধেক" নয়** — এটি মাইক্রোকার্নেলের মডুলারিটা নেয়, কিন্তু গতির জন্য ড্রাইভার কার্নেল স্পেসে রাখে। **মডুলার ≠ মাইক্রোকার্নেল**: মডিউল কার্নেল কোড, মাইক্রোকার্নেল সার্ভার ইউজার কোড। একটি **স্তর** কেবল ঠিক নিচের স্তরটি ব্যবহার করতে পারে।

মনে রাখার কৌশল: **"বড় কার্নেল, দ্রুত কল। ছোট কার্নেল, বকবক কল।"** আঁকার ধরন যাই হোক, কর্মক্ষমতা ঠিক করে একটিমাত্র বিষয়: অনুরোধ কি ইউজার-কার্নেল সীমানা পার হয়, নাকি একই অ্যাড্রেস স্পেসে থাকে।

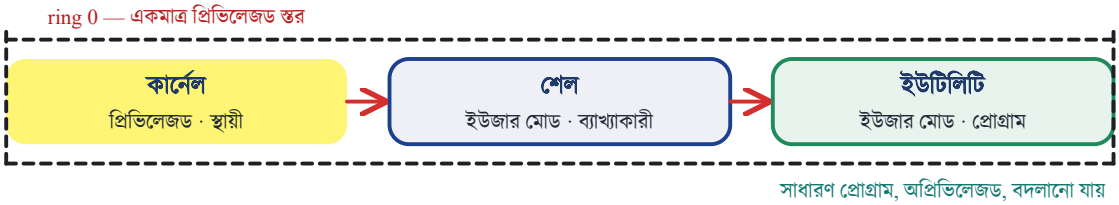
সমাধানকৃত উদাহরণ ✍ ছোট কার্নেলের দাম: একটি মাইক্রোকার্নেল একটি ফাইল রিড ক্লায়েন্ট → **IPC মেসেজ** → ফাইল সিস্টেম সার্ভার → **IPC মেসেজ** → ফেরত: প্রতি রাউন্ড ট্রিপে ২ µs হলে তা **৪ µs** মেসেজিং, যেখানে মনোলিথিক কার্নেল সরাসরি 0.1 µs-এর কমে কল করে। ১০০ ০০০ রিড/সে-এ CPU-র **৪০ %** মেসেজিংয়ে যায়; ১০০০ রিড/সে-এ মাত্র ০.৪ %।

বইয়ের উদাহরণ আসলে কোথায় বসে: **Windows NT** ও **macOS (XNU)** হাইব্রিড — macOS আসলে Mach মাইক্রোকার্নেল, তার চারপাশে BSD সেবা ও IOKit ড্রাইভার ফ্রেমওয়ার্ক। **Linux** মনোলিথিক, তবে *মডুলারভাবে বানানো*। **seL4** কয়েক হাজার লাইনের মাইক্রোকার্নেল যা **আনুষ্ঠানিকভাবে যাচাই** করা হয়েছে।

৪ তিনটি অংশ: কার্নেল, শেল, ইউটিলিটি

সংজ্ঞা: "অপারেটিং সিস্টেম" কথাটি দুটি ভিন্ন সফটওয়্যারের জগতের জন্য আলগোভাবে ব্যবহৃত হয়। সরু অর্থে এটি কেবল **কার্নেল**: প্রিভিলেজড, স্থায়ী, হার্ডওয়্যারের দায়িত্বে। বিস্তৃত অর্থে এটি কার্নেল + **শেল** + **ইউটিলিটি** — ব্যবহারকারী যা ছোঁয় এমন পুরো বিতরণ। তিনটি তিনটি ভিন্ন প্রিভিলেজ স্তরে বসে, আর "শেল কি OS-এর অংশ?" ধরনের প্রতিটি প্রশ্ন এই পার্থক্যেই আটকে থাকে।

> কার্নেল ≠ শেল ≠ ইউটিলিটি



	কার্নেল	শেল	ইউটিলিটি
যেখানে চলে	**কার্নেল মোডে**, x86 ring 0	ইউজার মোড, ring 3	ইউজার মোড, ring 3
মেমোরি	মূল মেমোরিতে স্থায়ী	একটি সাধারণ প্রসেস, বদলানো যায়	সাধারণ প্রসেস
কাজ	প্রসেস, মেমোরি, ফাইল, ডিভাইস, প্রোটেকশন	কমান্ড লাইন পড়ে, প্রোগ্রাম চালায়	একটি করে কাজ: *cp*, *ls*, *sort*, এডিটর
কী দিয়ে বদলায়	চলাকালে কখনো নয়	নির্দ্বিধায় — bash, sh, cmd.exe, PowerShell	নির্দ্বিধায় — যেকোনো প্রোগ্রামই
হার্ডওয়্যার ছোঁয়	**হ্যাঁ**	**না**	**না**

পরীক্ষার ফাঁদ: **শেল কার্নেল নয় এবং প্রিভিলেজড নয়**; এটি আপনার হয়ে সিস্টেম কল করে। শেল সরালে মেশিন ক্র্যাশ হবে না, কার্নেল নষ্ট হলে হবে। **কম্পাইলার** একটি ইউটিলিটি, OS-এর অংশ নয়।

মনে রাখার কৌশল: "**কার্নেলই সেটাই, যা বদলানো যায় না।**" কার্নেল — সরালে OS বন্ধ। শেল — আপনি নগ্ন লগইন প্রম্পটে আটকে যান। ইউটিলিটি — একটি কমান্ড হারিয়ে ফেলেন।

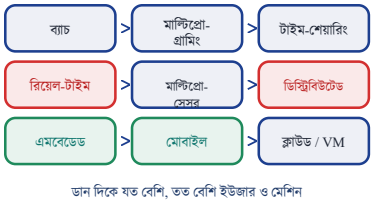
সমাধানকৃত উদাহরণ ✍️ একটি কী-প্রেস থেকে ফাইল পর্যন্ত:

cp a.txt b.txt: শেল লাইনটি ভাগ করে **execve()** দিয়ে *cp* চালায়; *cp* **open()**, **read()**, **write()**, **close()** করে, প্রতিটি ট্র্যাপ করে **কার্নেলে** ঢোকে। প্রিভিলেজড কোডে প্রবেশ: **পাঁচবার**।

বাস্তব নাম: শেল: UNIX-ধরনের সিস্টেমে **bash**, **zsh**, **fish**; Windows-এ **cmd.exe** ও **PowerShell**। ইউটিলিটি: **coreutils** পরিবার (*ls, cp, mv, grep*) এবং এডিটর ও কম্পাইলার। Linux-এ শেল `/bin`-এ একটি প্রোগ্রাম, ব্যবহারকারীভেদে বদলানো যায়।

৫ অপারেটিং সিস্টেমের প্রকার

সংজ্ঞা: একটি OS-কে "টাইপ" করা হয় **সে প্রসেসরকে কীভাবে ভাগ করে ও কাকে সেবা দেয়** তা দিয়ে, কোম্পানির নাম দিয়ে নয়। তিনটি প্রশ্নেই ধরা পড়ে: মেমোরিতে একসাথে কতটি কাজ, কোথাও কি কীবোর্ডের ইউজার আছে, ডেডলাইন কতটা কঠোর?



প্রতিটি বাক্স = একটি প্রতিশ্রুতি

	হার্ড রিয়েল-টাইম	সফট রিয়েল-টাইম
ডেডলাইন বাদ পড়লে	**ব্যর্থতা**	গুণের মান খারাপ হয়
উদাহরণ	পেসমেকার, ABS, ফ্লাইট কন্ট্রোল	স্ট্রিমিং ভিডিও, লাইভ টিভি
সাধারণ বিল্ড	ছোট ডেডিকেটেড RTOS	সাধারণ OS + RT সম্প্রসারণ

একই অক্ষ নয়: মাল্টিপ্রোগ্রামিং, টাইম-শেয়ারিং, মাল্টিপ্রোসেসর ও ডিস্ট্রিবিউটেড আলাদা করে *কতগুলো* মেশিন ও ইউজার; রিয়েল-টাইম ও এমবেডেড আলাদা করে *কোন প্রতিশ্রুতি* দেওয়া হয়। আপনার ফোন একই সাথে সবগুলো।

প্রকার	CPU কীভাবে ভাগ হয়	মূল লক্ষ্য	সাধারণ ব্যবহার
ব্যাচ	একবারে একটি কাজ, অফলাইনে সারিবদ্ধ	CPU-র ব্যবহার	মেইনফ্রেম চালানো
মাল্টিপ্রোগ্রামিং	অনেক কাজ মেমোরিতে; I/O-র অপেক্ষায় বদল	ব্যবহার	পরের দুটির ভিত্তি
টাইম-শেয়ারিং	অনেক ইউজার অনলাইন, ছোট স্লাইস	সাড়া + ন্যায্যতা	UNIX, Linux
মাল্টিপ্রোসেসর	দুই বা তত বেশি CPU ভাগ করে মেমোরি	সমান্তরালতা	সার্ভার
ডিস্ট্রিবিউটেড	স্বাধীন কম্পিউটার, এক মেশিন	ভাগাভাগি + নির্ভরযোগ্যতা	ক্লাস্টার
রিয়েল-টাইম	প্রায়োরিটি, সীমিত সাড়া	পূর্বানুমানযোগ্যতা	নিয়ন্ত্রণ, এভিওনিক্স
এমবেডেড	একটি নির্দিষ্ট চিপ, ক্ষুদ্র কার্নেল	নির্ধারিততা, কম পাওয়ার	গাড়ির ECU
মোবাইল	অ্যাপ স্যান্ডবক্স, ফ্ল্যাশ স্টোরেজ	ব্যবহারের অভিজ্ঞতা	Android, iOS

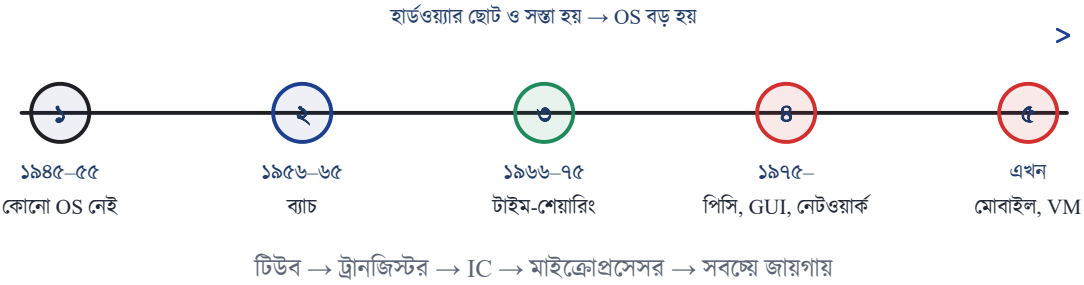
মনে রাখার কৌশল: **টাইম-শেয়ারিং মানুষের মধ্যে ভাগ করে; রিয়েল-টাইম ডেডলাইনের মধ্যে ভাগ করে।** টাইম-শেয়ার করা UNIX-ও ১ ms ডেডলাইন ছুড়তে পারে: ন্যায্যতা আর পূর্বানুমানযোগ্যতা বিপরীত দিকে টানে।

পরীক্ষার ফাঁদ: **রিয়েল-টাইম মানে দ্রুত নয়** — মানে **পূর্বানুমানযোগ্য**। ব্যাচ মানে "একক ব্যবহারকারী" নয়: একাধিক কাজকে একটি CPU-র সাথে ভাগ করাই মাল্টিপ্রোগ্রামিং।

সমাধানকৃত উদাহরণ ✍️ টাইম-শেয়ারিংের হিসাব:
১০০ জন, কোয়ান্টাম q: সর্বোচ্চ অপেক্ষা = ১০০ × q। **q = ১০০ ms → ১০ s; q = ২০ ms → ২ s**, তবে এখন সুইচ হয় প্রতি 0.2 ms-এ, তাই প্রতিটি ২ µs ধরলে CPU হারায় **১ %** সময়।

৬ অপারেটিং সিস্টেমের প্রজন্ম

সংজ্ঞা: অপারেটিং সিস্টেমকে তার **যে হার্ডওয়্যার প্রজন্মের জন্য বানানো হয়েছে** তা দিয়ে তারিখ দেওয়া হয়। পাঁচটি প্রজন্ম প্রমিত: প্রথমে **কোনো OS ছিল না**, আর প্রতিটি পরেরটি একটি নতুন সক্ষমতা কিনে আসে।



টিউব → ট্রানজিস্টর → IC → মাইক্রোপ্রসেসর → সবচেয়ে জায়গায়

প্রজন্ম	সময়	প্রযুক্তি	OS যা পেল	চিহ্ন
১ম	1945–1955	ভ্যাকুয়াম টিউব, পেপার টেইপ	**কিছু নয়**: একটি কাজ, মেশিন ভাষা	IBM 701, EDSAC (1948)
২য়	1956–1965	ট্রানজিস্টর	**ব্যাচ**: অফলাইন ইনপুট, স্পুলিং, নির্দিষ্ট কাজ	IBM 7090; FORTRAN 1957, COBOL 1959
৩য়	1966–1975	ইন্টিগ্রেটেড সার্কিট	**মাল্টিপ্রোগ্রামিং + টাইম-শেয়ারিং**	UNIX (1969), MULTICS, System/360 (1964)
৪র্থ	1975–	VLSI, মাইক্রোপ্রসেসর	পিসি যুগ: GUI, নেটওয়ার্ক, ক্লায়েন্ট–সার্ভার	MS-DOS (1981), Windows, Linux (1991)
৫ম	বর্তমান	অতি বড় স্কেল ইন্টিগ্রেশন, সর্বত্র ডিভাইস	মোবাইল, এমবেডেড, ভার্চুয়ালাইজড, ক্লাউড	Android (2008), হাইপারভাইজর

মনে রাখার কৌশল: **কিছু নয় → ব্যাচ → ভাগ করে ব্যবহার → GUI/নেটওয়ার্ক → মোবাইল/ক্লাউড।** প্রযুক্তিগুলো জোরে বলো — **টিউব, ট্রানজিস্টর, IC, মাইক্রোপ্রসেসর, সবচেয়ে জায়গায়** — তাহলে প্রজন্ম আর তারিখ নিজে থেকেই বেরিয়ে আসে।

পরীক্ষার ফাঁদ: প্রজন্ম **প্রকার নয়**: ব্যাচ ২য় প্রজন্মের একটি *প্রকার*, টাইম-শেয়ারিং ৩য় প্রজন্মের *প্রকার*। প্রশ্নে দুটি প্রতিদ্বন্দ্বী অপশন দিলে এই গুলিষ্টিই ধরা হচ্ছে।

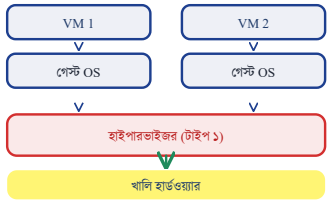
সমাধানকৃত উদাহরণ ✍️ সূত্রের সরল জেরে পড়া:
"মেমোরিতে অনেক কাজ, ইন্টারঅ্যাক্টিভ ইউজার, ইন্টিগ্রেটেড সার্কিট, বিখ্যাত 1969-এর সিস্টেম।" → **মাল্টিপ্রোগ্রামিং → টাইম-শেয়ারিং → তৃতীয় প্রজন্ম → UNIX**। সূত্র তারিখের চেয়ে জোরালো।

জানার মতো একটি ঐতিহাসিক বিবরণ: UNIX 1969 সালে DEC-এর **PDP-7**-এর জন্য Ken Thompson ও Dennis Ritchie লিখেছিলেন, 1970 সালে C-তে আবার লেখেন। প্রজন্ম-নাম *যুগ* বোঝায়, একটি নির্দিষ্ট মেশিন নয়।

৭ গঠনগত মডেল: এক্সটেনসিবল, VM ও হাইপারভাইজর

সংজ্ঞা: **এক্সটেনসিবল / VM-ভিত্তিক** মডেলে OS ছোট থাকে: কার্নেল কেবল ট্র্যাপ করে, শিডিউল করে ও সুইচ করে, আর প্রতিটি ইউজার তার উপরে একটি পুরো ছোট OS চালায় — সেটিই **ভার্চুয়াল মেশিন**। এতে ফল্ট আইসোলেশন পাওয়া যায়: একটি VM ডেউ হলে সিস্টেম নয়, শুধু সেই VM ডেউ হয়। আর এখান থেকেই এসেছে **হাইপারভাইজর** (আরেক নাম **VMM**)।

মডেল	ভাবনা	কে ব্যবহার করে
সিম্পল (MS-DOS-ধরন)	একটি প্রোগ্রাম, কোনো প্রোটেকশন নেই	MS-DOS, আগের CP/M
মনোলিথিক	প্রতিটি সেবা এক কার্নেল ব্লকে	UNIX, Linux
লেয়ারড	স্তর, প্রতিটি কেবল নিচেরটি ব্যবহার করে	THE, MULTICS, OS/2
মাইক্রোকার্নেল	ক্ষুদ্র কার্নেল, ইউজার-লেভেল সার্ভার	Mach, MINIX 3, QNX, seL4
মডুলার	কোর ছাড়া লোডেবল মডিউল	Linux, Solaris, VxWorks
হাইব্রিড	মাইক্রোকার্নেলের ভাবনা, মনোলিথিকের গতি	Windows NT, macOS XNU
এক্সটেনসিবল / VM	ক্ষুদ্র OS; প্রতিটি ইউজার পুরো OS চালায়	IBM VM/CMS, ES/VM; Xen, KVM



প্রতিটি গেস্ট মনে করে সে একা মালিক

বিষয়	টাইপ ১ — নেটিভ / বেয়ার-মেটাল	টাইপ ২ — হোস্টেড
চলে	**সরাসরি হার্ডওয়্যারের উপরে**	**হোস্ট OS-এর উপরে**, একটি অ্যাপ হিসেবে
হোস্ট OS লাগে?	না	হ্যাঁ — Windows, macOS, Linux
গতি	নেটিভের কাছাকাছি, পাতলা স্তর	ধীর: পথে হোস্ট কার্নেল
কে ব্যবহার করে	ডেটা সেন্টার, ক্লাউড, সার্ভার	ডেস্কটপ, ল্যাব, প্রশিক্ষণ
উদাহরণ	Xen, KVM, ESXi, Hyper-V, Proxmox	VirtualBox, Workstation, Parallels

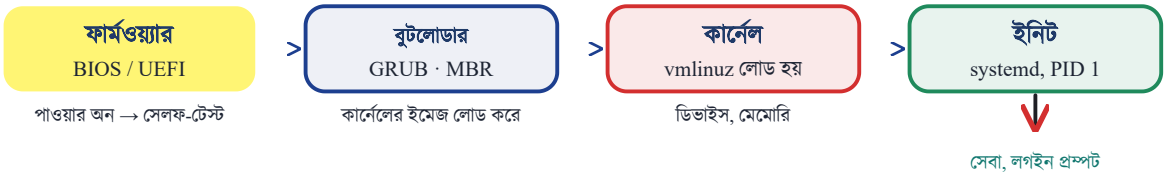
মনে রাখার কৌশল: "**টাইপ ১ মেঝে মালিক, টাইপ ২ বাড়িওয়ালার কাছ থেকে ভাড়া চায়।**" বিদ্যমান OS-এর *ভেতরে* বসানো হলে → টাইপ ২।

পরীক্ষার ফাঁদ: **হাইপারভাইজর** আর **কন্টেইনার** গুলিয়ে ফেলবেন না: কন্টেইনার হোস্ট **কার্নেল** ভাগ করে, VM-এ পুরো একটি গেস্ট OS থাকে।

সমাধানকৃত উদাহরণ ✍️
কনসলিডেশনের হিসাব:
২০টি পুরনো সার্ভার ২০ % লোডে, প্রতিটি অ্যাপ নিজ VM-এ: ২০ × ২০ % = এক কোরের ৪০০ %, মানে **৪টি কোর**। একটি 32-কোরের হোস্ট ১৫ % খালি রাখলে **১৯টি সার্ভার** বসানো যায়। **KVM** লিনাক্স কার্নেলকেই টাইপ ১ হাইপারভাইজর বানায়; **Xen** প্যারাভার্চুয়ালাইজড।

৮ বুট প্রসেস: ফার্মওয়্যার → বুটলোডার → কার্নেল → ইনিট

সংজ্ঞা: সংরক্ষিত প্রোগ্রাম মেশিন (stored-program machine) কোনো কিছু করে না, যতক্ষণ না কোনো ঠিকানায় একটি নির্দেশ থাকে আর প্রোগ্রাম কাউন্টার সেখানে দাঁড়ায়। **বুট সিকোয়েন্স** সেই সিঁড়ি, যা একজন মালিকের পর একজন মালিক হয়ে CPU-তে প্রথম নির্দেশ বসায় এবং মেশিনটি OS-এর হাতে দেয়।



প্রতিটি ধাপে ঠিক একটি প্রোগ্রামই দায়িত্বে

প্রত্যেকটি পরেরটিকে লোড করে, তারপর সরে যায়

ধাপ	এটি কী করে	কোথায় থাকে	উদাহরণ
১. ফার্মওয়্যার	পাওয়ার-অন সেলফ-টেস্ট, হার্ডওয়্যার চালু, বুট ডিভাইস খোঁজে	মেসিনবোর্ডের একটি চিপ; CMOS-তে সেটিংস	পুরনো **BIOS**; আধুনিক **UEFI**
২. বুটলোডার	কার্নেলের ইমেজ RAM-এ নিয়ে সেখানে লাফ দেয়	ডিস্কের প্রথম সেক্টর — **MBR** বা **ESP** পার্টিশন	GRUB, Windows Boot Manager, U-Boot
৩. কার্নেল	সাবসিস্টেম চালু করে, মেমোরি সাজায়, ডিভাইস খোঁজে, **ইনিট** শুরু করে	RAM-এ লোড, এখন স্থায়ী	Linux, Windows NT, BSD
৪. init / systemd	ক্রমে সেবা চালায়, ফাইল সিস্টেম মাউন্ট করে, লগইন ম্যানেজার চালায়	প্রথম ইউজার-স্পেস প্রসেস, **PID 1**	systemd (আধুনিক Linux), SysV init, launchd

মনে রাখার কৌশল: **"ফার্মওয়্যার মেশিন বিক্রি করে, বুটলোডার OS তোলে, কার্নেল মেশিনের মালিক হয়, ইনিট সেবা চালায়।"** চারটি ক্রিয়া, চারজন মালিক — কেবল ধাপ ৩ *কার্নেল*।

পরীক্ষার ফাঁদ: **বুটলোডার OS নয়**, আর **BIOS ফার্মওয়্যারের একমাত্র নাম নয়** — UEFI তার উত্তরসূরী, এবং এটি MBR নয়, ESP ব্যবহার করে। **কার্নেল প্যানিক** মানে ধাপ ৩-এ ব্যর্থতা।

সমাধানকৃত উদাহরণ ✍️ পরীক্ষকী কেন জিজ্ঞেস করেন:
"লোগো, আর কিছু নয়।" POST শব্দ না হলে বুটলোডারের **আগেই** ব্যর্থতা (RAM, CPU, পাওয়ার)। "No bootable device" → ফার্মওয়্যার চলেছে, কিন্তু বুটযোগ্য ডিভাইস পায়নি। GRUB মেনু নেই → বুটলোডার নষ্ট।

IT সংযোগ: ডুয়াল বুটেই বুটলোডার নিজের কাজ দেখায়: GRUB একটি মেনু দেখায় ও অন্য কার্নেল লোড করে — তাই এটি কোনো OS-এর অংশ নয়, এবং একই বুটলোডার একই ডিস্ক থেকে Windows ও Linux দুটোই চালাতে পারে।

৯ ট্র্যাপ, ইন্টারাপ্ট ও কনটেক্সট সুইচের খরচ

সংজ্ঞা: কার্নেলে নিয়ন্ত্রণ আসে তিনভাবে। **ট্র্যাপ** সিঙ্ক্রন — প্রোগ্রাম নিজেই ঘটিয়েছে (সিস্টেম কল, পেজ ফল্ট, শূন্যে ভাগ)। **ইন্টারাপ্ট** অ্যাসিঙ্ক্রন — ডিভাইস ঘটিয়েছে (টাইমার, কীপ্রেস, ডিস্ক)। **ফল্ট** হলো CPU কুনড় নির্দেশের রিপোর্ট করছে। সবাই কনটেক্সট সেভ করে, ভেক্টর টেবিল থেকে একটি **ভেক্টর** নিয়ে কার্নেল মোডে হ্যান্ডলার চালায়।

প্রকার	কে ঘটায়	কখন	উদাহরণ
ট্র্যাপ	চলতে থাকা প্রোগ্রাম	সিঙ্ক্রন	read(), exit(), ব্রেকপয়েন্ট
ফল্ট	নির্দেশটি নিজেই	সিঙ্ক্রন	পেজ ফল্ট, শূন্যে ভাগ
ইন্টারাপ্ট	বাইরের একটি ডিভাইস	অ্যাসিঙ্ক্রন	টাইমার টিক, কীবোর্ড, ডিস্ক



সময় ধার্য হয় যেখানে ট্র্যাপ নেওয়া হয়

কাজ	খরচ	সময় কোথায় যায়
ফাংশন কল	≈ ১–১০ ns	রেজিস্টার সেভ, লাফ, ফেরা; ক্যাশ গরম থাকে
সিস্টেম কল (ট্র্যাপ)	≈ ১ µs	কনটেক্সট সংরক্ষণ, কার্নেল মোড, ডিসপ্যাচ, ফেরা
কনটেক্সট সুইচ	≈ **১–১০ µs**	PCB সেভ ও রিস্টোর; পরে TLB ও ক্যাশ ঠান্ডা
মেজর পেজ ফল্ট	≈ ১০০ µs – ১ ms	অনুপস্থিত পেজের জন্য ডিস্ক রিড

কনটেক্সট সুইচের আসল খরচ: একটি প্রসেসের CPU অবস্থা সেভ করে আরেকটি রিস্টোর করা কেবল **প্রত্যক্ষ** খরচ, সাধারণত **১–১০ µs**। **পরোক্ষ** খরচ আরও বড়: নতুন প্রসেস পায় খালি ইনস্ট্রাকশন ক্যাশ, ঠান্ডা TLB ও ঠান্ডা ডেটা ক্যাশ। বেশি সিস্টেম কল বা ছোট কোয়ান্টাম মানে বেশি সুইচ।

মনে রাখার কৌশল: "**ট্র্যাপ প্রোগ্রামের দোষ, ইন্টারাপ্ট পৃথিবীর।**"

পরীক্ষার ফাঁদ: একটি **পেজ ফল্ট ক্র্যাশ নয়** — OS ফ্রেম পেজ করে নির্দেশটি আবার চালায়। আর **ট্র্যাপ কনটেক্সট সুইচ নয়**: তা মানে *অন্য* কোনো প্রসেস চলেছিল।

সমাধানকৃত উদাহরণ ✍️ সুইচে হারানো থ্রুপুট:
৫০ জন, কোয়ান্টাম **২০ ms**, সুইচ **২ µs**। গড় অপেক্ষা ≈ (৫০ − ১)/২ × ২০ ms ≈ **৪৯০ ms**। প্রতি স্লাইসে ওভারহেড = ২/২০ ০০০ = **০.০১ %** — এটাই নকশার ধারা।

IT সংযোগ: `time` একটি প্রোগ্রামের জীবনকে **ইউজার** ও **সিস্টেম** সময়ে ভাগ করে; সিস্টেম সময় মানে কার্নেলে কাটানো সময়। CPU-র ব্যবহার কম থাকা অথচ সিস্টেম সময় বেশি হলে তা **থ্র্যাশিং**-এর চিহ্ন।

রাতের-পরীক্ষার পাতা

তথ্য-পত্র:

OS = ইন্টারফেস + রিসোর্স ম্যানেজার + কন্ট্রোল প্রোগ্রাম

লক্ষ্য: সুবিধা, দক্ষতা, বিবর্তন, পোর্টেবিলিটি

কার্নেল = প্রিভিলেজড, স্থায়ী, সবসময় RAM-এ

শেল = ইউজার মোডের কমান্ড ব্যাখ্যাকারী

সিস্টেম কল ⇒ ট্র্যাপে ইউজার → কার্নেল মোড

x86: ইউজার = ring 3, কার্নেল = ring 0

মনোলিথিক: সব কার্নেল স্পেসে (UNIX, Linux)

মাইক্রোকার্নেল: শুধু IPC, শিডিউলিং, অ্যাড্রেস স্পেস

হাইব্রিড: Windows NT, macOS XNU

মডুলার: Linux, Solaris

টাইপ ১ হাইপারভাইজর = বেয়ার মেটাল

টাইপ ২ = হোস্ট OS-এর উপরে (VirtualBox)

বুট: ফার্মওয়্যার → বুটলোডার → কার্নেল → ইনিট

init / systemd হলো PID 1

ট্র্যাপ সিঙ্ক্রন · ইন্টারাপ্ট অ্যাসিঙ্ক্রন

কনটেক্সট সুইচ ≈ ১–১০ µs; কল ≈ ২০ ns

MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. OS = একটি ইন্টারফেস **এবং** একটি রিসোর্স ম্যানেজার
2. চারটি লক্ষ্য: সুবিধা, দক্ষতা, বিবর্তন, পোর্টেবিলিটি
3. কার্নেল = প্রিভিলেজড কোর; শেল = কেবল একটি UI
4. সিস্টেম কল ট্র্যাপের মাধ্যমে ইউজার → কার্নেল মোডে যায়
5. মনোলিথিক: ফাইল সিস্টেম ও ড্রাইভার কার্নেল স্পেসে
6. মাইক্রোকার্নেল: শুধু IPC, শিডিউলিং, অ্যাড্রেস স্পেস
7. হাইব্রিড কার্নেল = Windows NT, macOS XNU
8. টাইপ ১ বেয়ার মেটালে; টাইপ ২ হোস্ট OS-এর উপরে
9. বুট: ফার্মওয়্যার → বুটলোডার → কার্নেল → ইনিট
10. init (systemd) প্রথম প্রসেস, PID 1
11. ট্র্যাপ সিঙ্ক্রন, ইন্টারাপ্ট অ্যাসিঙ্ক্রন
12. রিয়েল-টাইম মানে দ্রুত নয়, পূর্বানুমানযোগ্য

সবচেয়ে বড় ফাঁদগুলো: শেল কার্নেল নয় · একটি ড্রাইভার একটি ডিভাইস সামলায়, OS সবগুলো · প্রজন্ম প্রকার নয় · রিয়েল-টাইম "দ্রুত" নয় · পেজ ফল্ট ক্র্যাশ নয় · ট্র্যাপ কনটেক্সট সুইচ নয় · কন্টেইনার VM নয়।

সব কৌশল একসাথে: চারটি G: সুবিধা = সহজ, দক্ষতা = পূর্ণ, বিবর্তন = বদলানোযোগ্য, পোর্টেবিলিটি = বাড়ি বদলায় · কার্নেল বদলানো যায় না · ট্র্যাপ = প্রোগ্রামের দোষ, ইন্টারাপ্ট = পৃথিবীর · বড় কার্নেল = দ্রুত কল, ছোট কার্নেল = বকবক কল · টাইপ ১ মেঝের মালিক, টাইপ ২ বাড়িওয়ালার কাছে চায়।



সব কিছু মাঝের তিরের উপর নির্ভরশীল

সমাধানকৃত উদাহরণ ✍️ একটি সংখ্যাই উত্তর বদলে দেয়: মাইক্রোকার্নেলে ২ µs × ২ মেসেজ = ৪ ms/s = **০.৪ %** ওভারহেড প্রতি ১০০০ অনুরোধ/সে-এ; ১০০ ০০০ অনুরোধ/সে-এ তা ৪০ %। একই গঠন, ভিন্ন উত্তর — গঠন-প্রশ্ন আসলে লোড-প্রশ্ন।

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

১. OS-এর কোন অংশটি স্থায়ী থাকে এবং CPU, মেমোরি ও ডিভাইস সামলায়?
ক) শেল খ) কার্নেল
গ) কম্পাইলার ঘ) বুটলোডার

২. সিস্টেম কল কী:
ক) একটি হার্ডওয়্যার ফল্ট খ) একটি প্রোগ্রামের ভেতরের কল
গ) কার্নেলের সেবা পাওয়ার ইন্টারফেস ঘ) মেশিন রিস্টার্টের কমান্ড

৩. কোন গঠনে কার্নেলে কেবল IPC, শিডিউলিং ও নিম্নমানের মেমোরি থাকে, আর ফাইল ও ড্রাইভার ইউজার সার্ভার হয়?
ক) মাইক্রোকার্নেল খ) মনোলিথিক
গ) লেয়ারড ঘ) সিম্পল (MS-DOS-ধরন)

৪. ইউজার প্রোগ্রামকে কার্নেলের সেবায় পৌঁছানোর প্রিভিলেজ পরিবর্তনটি কী:
ক) কনটেক্সট সুইচ খ) ডিসপ্যাচ
গ) ট্র্যাপ ঘ) পেজ রিপ্লেসমেন্ট

৫. ২ ms-এর মধ্যে হৃদস্পন্দন উদ্দীপিত করতে হবে এমন পেসমেকার চলে কোনটিতে?
ক) ব্যাচ OS খ) হার্ড রিয়েল-টাইম OS
গ) টাইম-শেয়ারিং OS ঘ) ডিস্ট্রিবিউটেড OS

৬. কোন প্রজন্মে ছিল ভ্যাকুয়াম টিউব, কোনো OS নয় এবং মেশিন ভাষা?
ক) প্রথম খ) দ্বিতীয়
গ) তৃতীয় ঘ) চতুর্থ

৭. কোনো হোস্ট OS ছাড়াই সরাসরি হার্ডওয়্যারের উপর বসানো হাইপারভাইজর কী:
ক) টাইপ ১ খ) টাইপ ২
গ) টাইপ ৩ ঘ) একটি কন্টেইনার রানটাইম

৮. স্টার্টআপে কার্নেলের ইমেজ মেমোরিতে কে লোড করে?
ক) শেল খ) বুটলোডার
গ) ইনিট ঘ) ডিভাইস ড্রাইভার

৯. একটি পেজ ফল্ট কী:
ক) এমন ক্র্যাশ যা প্রসেস শেষ করে দেয় খ) অ্যাসিঙ্ক্রন ডিস্ক ইন্টারাপ্ট
গ) একটি সিঙ্ক্রন ফল্ট, OS সেবা দিয়ে নির্দেশটি আবার চালায় ঘ) শিডিউলারের কনটেক্সট সুইচ

১০. Linux-এ প্রথম ইউজার-স্পেস প্রসেস (PID 1) কে:
ক) বুটলোডার খ) শেল
গ) init / systemd ঘ) ফাইল সিস্টেম

উত্তরমালা → ১-খ · ২-গ · ৩-ক · ৪-গ · ৫-খ · ৬-ক · ৭-ক · ৮-খ · ৯-গ · ১০-গ

লিখিত প্রশ্ন: অপারেটিং সিস্টেমের সংজ্ঞা দাও, সেটিকে রিসোর্স ম্যানেজার ও কন্ট্রোল প্রোগ্রাম হিসেবে ব্যাখ্যা করো, চিহ্নিত চিত্রে দেখাও কীভাবে ইউজার প্রোগ্রাম সিস্টেম কলের মাধ্যমে কার্নেলের সেবায় পৌঁছায়, এবং মনোলিথিক ও মাইক্রোকার্নেল গঠনের তুলনা করো। ১০ নম্বর

১. **সংজ্ঞা (২)** — ব্যবহারকারী ও হার্ডওয়্যারের মাঝের সিস্টেম সফটওয়্যার, যা CPU, মেমোরি, ফাইল ও ডিভাইস সামলায়। ২

২. **রিসোর্স ম্যানেজার (২)** — *কে কখন কী পাবে*: CPU শিডিউলিংয়ে, মেমোরি অ্যালোকেশন ও পেজিংয়ে, ফাইল ফাইল সিস্টেমে, ডিভাইস ড্রাইভারে। ২

৩. **কন্ট্রোল প্রোগ্রাম (১)** — কেবল এটিই হার্ডওয়্যার ছোঁয়; প্রোগ্রামগুলো আলাদা থাকে, তাই একটির বাগ পুরো সিস্টেম ক্ষতিগ্রস্ত করতে পারে না। ১

৪. **কলের পথ (৩)** — প্রোগ্রাম → লাইব্রেরি রুটিন → **ট্র্যাপ** → CPU কনটেক্সট সেভ করে **কার্নেল মোডে** যায় → সিস্টেম-কল টেবিল থেকে ডিসপ্যাচ → সেবা → return-from-trap। চিহ্নিত করো: ইউজার মোড, কার্নেল মোড, ট্র্যাপ, হার্ডওয়্যার। ৩

৫. **তুলনা (২)** — **মনোলিথিক**: সব সেবা এক কার্নেলে, দ্রুত কিন্তু একটি বাগ মেশিন থামাতে পারে (UNIX, Linux)। **মাইক্রোকার্নেল**: কেবল IPC, শিডিউলিং ও অ্যাড্রেস স্পেস; ফাইল ও ড্রাইভার ইউজার সার্ভার — নির্ভরযোগ্য, কিন্তু মেসেজ ওভারহেড বেশি (Mach, QNX)। ২

পরীক্ষার আগে ✓

- ✓ চারটি লক্ষ্য বলতে পারি
- ✓ কার্নেল, শেল, ইউটিলিটির নাম বলতে পারি
- ✓ ট্র্যাপের পথ আঁকতে পারি
- ✓ আটটি প্রকারের নাম বলতে পারি
- ✓ পাঁচটি প্রজন্ম স্থানে বসাতে পারি
- ✓ টাইপ ১ ও টাইপ ২ আলাদা করতে পারি